Data Science Program

Capstone Report – Fall 2025

Enhancing Residential Property Valuation through Latent Manifold Discovery and Spatiotemporal Multi-Task Quantile Regression Frameworks

Ujjawal Dwivedi

Supervised by Prof. Amir Jafari

The George Washington University

Washington, DC

Abstract

We develop a two–component neural framework for house price prediction that operates jointly at the individual property level and the ZIP–by–month market level. First, we implement a Latent Manifold Estimation (LME) model that decomposes the log price of each home into an intrinsic value—predicted from structured property attributes via a neural network—and a smooth spatial desirability surface learned over geographic coordinates. Second, we construct an end–to–end spatio–temporal pipeline that transforms property–level transaction histories into a panel of ZIP–by–month price indices and trains a tabular multi–task quantile network (TAB–MTL) to forecast one– and two–month–ahead index returns with calibrated predictive intervals. Taken together, these components deliver market–level forecasts for each ZIP code and granular adjustments for individual homes, providing a practical blueprint for data–driven pricing, risk assessment, and land acquisition strategy for home builders and institutional investors

Contents

1 Introduction

1.1 Motivation and business context

Residential real estate is one of the largest asset classes in the world. For home builders, developers, and institutional investors, small improvements in pricing and timing decisions can translate into millions of dollars in upside or avoided downside. For individual buyers, these same decisions determine long–term affordability and household wealth. Yet, house prices are driven by a complex interaction of factors operating at very different spatial and temporal scales:

- **Macro and meso–level drivers** such as mortgage rates, unemployment, and broader economic conditions.
- **Local market dynamics** such as ZIP–level inventory, demand pressure, and neighborhood reputations.
- **Micro–level heterogeneity** at the individual property level: square footage, layout, age, finishes, school district, and even which side of a street the home sits on.

Traditional statistical models tend to address either the aggregate (e.g., ZIP–level price indices) or the micro level (hedonic regressions on individual properties), but not both in an integrated framework. This capstone proposes a two–component neural system that explicitly separates these scales:

1. A **ZIP×month forecasting model** that learns the temporal dynamics of aggregated price indices and produces calibrated prediction intervals for future months.
2. A **Latent Manifold Estimation (LME) model** that explains fine–grained cross–sectional variation within a ZIP by learning an intrinsic structural value for each home plus a smooth "desirability field" over geographic space.

Together, these components support practical questions that a home builder or investor might ask:

- *"Where are prices in this ZIP likely to be in 1–2 months?"*
- *"Given that forecast, what is a fair price for this particular house on this block?"*

1.2 High–level approach

Our starting point is a large, web–scraped dataset of residential properties that resembles public portals such as Zillow. Each property (identified by a Zillow–style ID `ZPID`) has:

- detailed home attributes (beds, baths, square footage, lot size, year built, type);
- geographical identifiers (latitude/longitude, ZIP code, county and state codes);
- a rich `PRICEHISTORY` JSON, which records historical listing and sale events;
- auxiliary market indicators (days on market, hotness scores, mortgage rate, unemployment).

From this raw source, we construct two related modeling pipelines:

Component I – Latent Manifold Estimation: We model the log price of each home as the sum of (i) an *intrinsic* component predicted by a neural network based on tabular attributes, and (ii) a *desirability field* over geographic coordinates. The desirability term is represented by a vector D with one scalar per training home, and we interpolate it via a kernel surface. A matrix–free Conjugate Gradient solver with Laplacian regularization is used to estimate D, leading to a smooth latent manifold of neighborhood effects.

Component II – ZIP×Month TAB–MTL Quantile Model: We flatten the JSON event histories into an event–level table, then aggregate to ZIP×month to build a robust price index. Using time–series features, approximate H3 geo–tiling, macroeconomic lags, and seasonality, we train a tabular neural network in a multi–task quantile–regression setting. This network jointly forecasts one–month–ahead (H1) and two–month–ahead (H2) index returns and outputs prediction intervals via multiple quantiles.

The LME component operates at the *individual property* level and is primarily cross–sectional, whereas the TAB–MTL model operates at the *ZIP×month panel* level and is explicitly temporal. The two are conceptually linked: in a production setting the ZIP index forecast would provide a baseline level for each market, while LME provides a house–specific deviation from that ZIP trajectory.

1.3 Contributions and proposal

The main contributions of this capstone are:

- **Unified data engineering pipeline:** from raw JSON price histories to an event–level table, to ZIP×month indices, and finally to tabular feature matrices for neural modeling. This pipeline is implemented with an eye toward scalability and reproducibility.
- **Implementation of Latent Manifold Estimation:** a faithful implementation of the LME framework, including kernel–based interpolation of neighborhood desirability, matrix–free graph Laplacian regularization, and an EM–style training loop that alternates between updating the desirability vector D and the intrinsic neural network.
- **Multi–task neural ZIP index forecaster:** A tabular quantile network that learns to forecast both H1 and H2 horizons jointly with shared embeddings and trunk layers, while using reliability weighting to focus learning on well–observed ZIP×month cells.
- **Integrated view of pricing:** A combined narrative that connects macro ZIP–level forecasts with micro property–level valuations, illustrating how such a stacked system could be deployed by a builder or institutional investor to support pricing, land acquisition, and risk management decisions.

From a practical standpoint, we are proposing a blueprint for a *Neural Zestimate–style stack* tailored to a homebuilder:

1. Use the ZIP×month quantile model to update market–level price forecasts on a regular schedule (e.g., monthly).
2. Use the LME model, trained on recent transactions, to assign a fair value to each candidate property relative to its peers, decomposed into intrinsic structure and neighborhood desirability.

3. Combine these in a decision engine that flags mispriced deals, quantifies upside/downside scenarios, and provides interpretable drivers of value at both the market and street level.

The remainder of the report is organized as follows. Section 2 describes the shared dataset and preprocessing steps. Section 3 details the Latent Manifold Estimation model, including geometry, kernel construction, and the EM optimization scheme. Section 4 presents the ZIP×month index construction and the TAB–MTL quantile forecasting model, together with evaluation metrics and results. Section 5 concludes with a combined discussion and directions for future work.

2 Data and Preprocessing (Shared Foundations)

The starting point for the ZIP×Month component is a CSV export (here called `sub_sample.csv`) of a larger scraped dataset derived from online real–estate listings. Each row represents a snapshot of a property (identified by a Zillow–style ID `ZPID`) at a particular scrape time. The same underlying dataset can be used as the basis for both the latent manifold and ZIP–level models, with the manifold model operating at individual property level and the TAB–MTL model operating on aggregated ZIP×month indices.

2.1 Raw dataset

Each row of the base CSV contains the following key groups of columns:

- **Identifiers and location.** ZPID, street address, city, state, county, ZIP code, FIPS and county codes, latitude and longitude.
- **Home attributes.** Bedrooms, bathrooms, square footage, lot size, year built, home type, construction materials, presence of pool, parking, etc.
- **School and neighborhood attributes.** Distances and ratings for elementary, middle and high schools; demographic and economic statistics by ZIP (population, income, median age, growth metrics).
- **Market indicators.** Days on market, median listing price, hotness score, supply and demand scores, weekly average mortgage rate, and unemployment rate.
- **History JSON.** A `PRICEHISTORY` column containing a JSON array of past events for that property (listing, price change, sold, pending, etc.), with timestamps and prices.

In the offline experiment used in this report, the raw CSV contains approximately 8.6×10^5 rows and 150+ columns. After flattening the JSON history (Section 4.1) we obtain roughly 5.0×10^5 price events from January 2022 onward.

2.2 Downcasting and type normalization

For memory efficiency, all numeric columns are downcast to the smallest feasible numeric dtype:

- floating–point columns are converted to `float32`;
- integer columns are downcast to `int32` or `int16` when possible;
- object columns with moderate cardinality (less than 40% unique values) are cast to `category`.

Helper functions are used to coerce messy text representations into numeric values. For example, `safe_to_double` strips commas, currency symbols and percent signs from strings like "$1,234" or "5%" before parsing them into floats. Boolean flags are normalized using `safe_to_binary_from_text` and `safe_to_binary_from_number`. This ensures that subsequent grouping and aggregation operations in `pandas` are memory–efficient and numerically stable.

3 Component I: Latent Manifold Estimation for Individual House Pricing

3.1 Intuition and model overview

Homes with similar structural attributes can still sell for very different prices because of neighborhood desirability: school clusters, street micro–amenities, noise, safety, walkability, and countless unobserved local features. A purely tabular model that uses only structured attributes will typically encode these effects in opaque combinations of latitude/longitude and ZIP dummies, making it hard to interpret and sometimes unstable geographically.

To address this, we explicitly decompose the log–target into an intrinsic (structure–driven) term and a spatially–smoothed desirability term.

For property $i \in \{1, \ldots, N\}$ we observe $x_i^{(\text{param})} \in \mathbb{R}^d$ (tabular features), $s_i \in \mathbb{R}^2$ (lat/lon), non–negative weight w_i, and log–target y_i. We posit the additive model

$$y_i = m_i + h_i + \varepsilon_i, \qquad m_i = G(x_i^{(\text{param})}; W), \quad h_i = H(s_i; D), \tag{1}$$

where G is a BN+ReLU MLP parameterized by W and H is a nonparametric spatial smoother controlled by $D \in \mathbb{R}^N$. The vector D holds one scalar "desirability" value per train home; at inference, a new location interpolates h^* from its neighbors' d_j values.

Intuitively, G learns how house structure and coarse location affect price, while H captures local premiums and discounts that vary smoothly in space. This mirrors how a human appraiser might reason: start with a baseline price from bed/bath/sqft and then adjust up or down for micro–location.

3.2 Variable glossary (symbols ↔ code)

Table 1 maps the mathematical symbols to code handles and their meaning.

3.3 Geometry: projecting lat/lon and standardization

Raw latitude and longitude are expressed in degrees on a sphere, which is not ideal for Euclidean distance calculations. We therefore convert degrees to a local metric plane centered at the train mean latitude $\bar{\phi}$:

$$x_i = R(\lambda_i - \bar{\lambda}) \cos \bar{\phi}, \quad \bar{y}_i = R(\phi_i - \bar{\phi}), \quad R = 6.371 \times 10^6 \text{ m}. \tag{2}$$

The surface coordinates $[y_i, x_i]$ are then standardized by train mean/std (so KDTree distances are isotropic). Code: `latlon_to_xy_meters`, standardized by train.

Symbol	Code handle	Meaning / Effect
$x_i^{(\text{param})}$	X_tr[i]	Standardized param features for home i; input to MLP.
$s_i = [\text{lat}, \text{lon}]_i$	S_tr_std[i]	Standardized meters (Eq. 2); drives spatial neighbors.
y_i	y_tr[i]	Log target: log(PPSQFT) if sqft available, else log(price).
w_i	w_tr[i]	Sample weight (upweights higher PPSQFT quantiles).
W	`model.state_dict()`	MLP parameters of the intrinsic function G.
D	`trainer.D`	Desirability vector; one scalar per train row; produces the surface via ne
K	`hp.K`	# neighbors in KDTree; larger K = smoother / less variance, more bias
q	`hp.q`	Kernel sharpness; larger q decays weights faster, reducing spatial bleed.
r	`hp.reg_r`	Ridge on D; higher r shrinks D to zero, shifting burden to MLP.
λ	`hp.lap_lambda`	Laplacian smoothing weight; controls spatial coherence of D.
L	`LaplacianOp`	Matrix–free Laplacian operator (Eq. 8).
$\mathcal{N}(i)$	KDTree query	The K nearest neighbors of i in standardized meters.
w_{ij}	`KernelSurface`	Adaptive kernel weights (Eq. 3); $\sum_{j\in\mathcal{N}(i)} w_{ij} = 1$.
U_i	implicit	Sparse vector with $(U_i)_j = w_{ij}$ if $j \in \mathcal{N}(i)$ else 0.

Table 1: Symbols and their roles with code references in the LME implementation.

3.4 Kernel surface and interpolation

For each train point i, KDTree returns neighbors $\mathcal{N}(i)$ and distances d_{ij}. We use an adaptive bandwidth

$$\sigma_i^2 = \text{median}\{d_{ij}^2\} + 10^{-12}$$

and define

$$w_{ij} \propto \exp\left(-\frac{q d_{ij}^2}{2\sigma_i^2}\right), \quad \sum_{j\in\mathcal{N}(i)} w_{ij} = 1. \tag{3}$$

The surface values are

$$h_i = \sum_{j\in\mathcal{N}(i)} w_{ij} d_j \quad \text{(train)}, \tag{4}$$

$$h(s^*; D) = \sum_{j\in\mathcal{N}(*)} \tilde{w}_j^* d_j \quad \text{(inference)}. \tag{5}$$

Code: `KernelSurface.build_U_list`, `KernelSurface.interpolate_one`.

3.5 Objective and EM decomposition

We minimize the energy

$$E(W, D) = \frac{1}{2}\sum_{i=1}^{N} w_i\big(y_i - m_i - h_i\big)^2 + \frac{r}{2}\|D\|_2^2 + \frac{\lambda}{2} D^\top L D. \tag{6}$$

With G fixed, E is quadratic in D; with D fixed, it is a standard weighted MSE for the MLP trained on residual targets. This naturally leads to an EM–style algorithm.

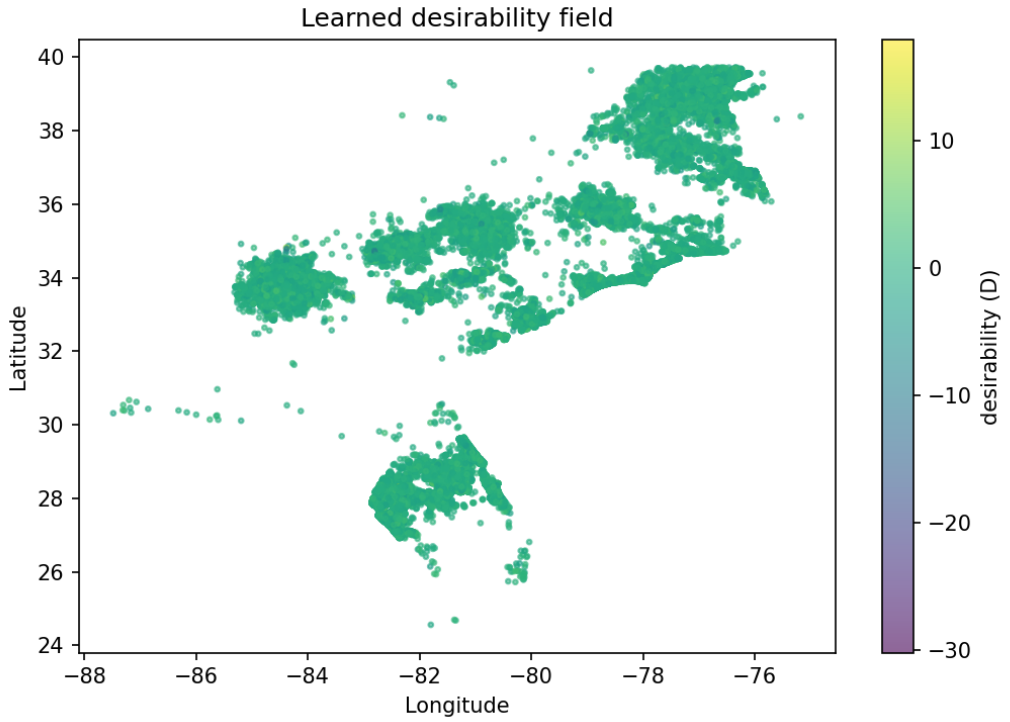

Figure 1: Learned desirability field D over the training geography. Colors indicate neighborhood premiums and discounts after controlling for structural attributes via the intrinsic neural network.

E–step: solving for D

Let U_i be the sparse weight vector for point i (Eq. 3). The normal equations are

$$\left(rI + \sum_{i=1}^{N} w_i U_i U_i^\top + \lambda L\right) D = \sum_{i=1}^{N} w_i (y_i - m_i) U_i. \tag{7}$$

We solve (7) using matrix–free Conjugate Gradient: given v, we compute

$$Av = rv + \sum_i w_i (U_i^\top v) U_i + \lambda L v$$

in $O(NK)$ time without forming A. Early stopping uses (i) a relative residual threshold and (ii) objective plateau patience. Finally we apply a gauge correction

$$D \leftarrow D - \frac{1}{N} \sum_i d_i$$

to keep the decomposition $m + h$ identifiable. Code: `LMETrainer.update_D`, `cg_solve_qp`.

Intuitively: rI shrinks D to stabilize ill–conditioned neighborhoods; $\sum_i w_i U_i U_i^\top$ enforces that D agrees with (kernel–averaged) residuals at each point; λL encourages spatial smoothness (neighbors should have similar desirability).

M–step: training G on residuals

With D fixed, we form residual targets

$$y_i^{(\text{res})} = y_i - h_i$$

and train the MLP with AdamW (learning rate warmup + cosine schedule) and an inner spatial validation split for early stopping. Code: `LMETrainer.mstep`.

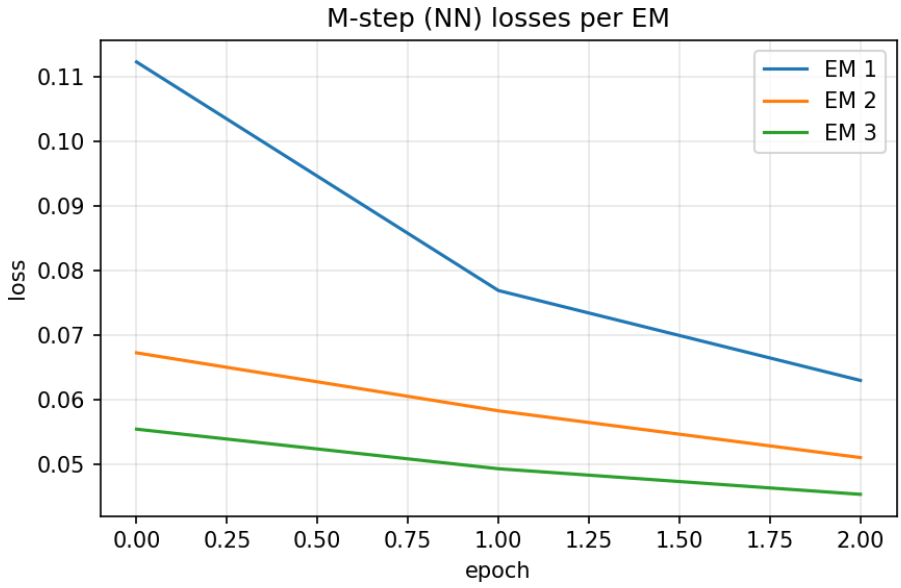

Figure 2: M–step neural network losses across epochs for successive EM iterations in the LME model. Each curve corresponds to one EM round, showing monotone improvement of the intrinsic value network as the desirability field is updated.

3.6 Graph Laplacian (optional)

Using the same KDTree, we define

$$(Lv)_i = \sum_{j \in \mathcal{N}_L(i)} a_{ij}(v_i - v_j), \quad a_{ij} \propto \exp\left(-\frac{d_{ij}^2}{2\tilde{\sigma}_i^2}\right), \tag{8}$$

so that $D^\top L D$ penalizes roughness of D over the neighbor graph. Code: `LaplacianOp.apply`.

3.7 Data pipeline and splits (LME)

Preprocessing for LME: load raw CSV; clean/engineer features; prepare matrix X, target y, and spatial columns. Standardize X and surface coordinates by train statistics only.

Spatial TRAIN/TEST: a coarse 100×100 grid over degrees is used to pick disjoint regions (reduces leakage). Inner VAL: a second 60×60 grid inside TRAIN for M–step early stopping. Code: `DataPreprocessor`, spatial grid split, `make_inner_spatial_val`.

3.8 Architecture and training details

- **Intrinsic network:** MLP with hidden sizes $(256, 128, 64, 32)$, BatchNorm+ReLU, dropout $p = 0.25$.
- **Optimizer:** AdamW, learning rate 5×10^{-4}, weight decay 5×10^{-4}, batch size 512.
- **LR schedule:** linear warmup to epoch E_{warm} then cosine decay within the M–step.
- **CG:** tolerance 10^{-5}, max 300 iterations, patience 10; complexity per matvec $O(NK)$.
- **Regularization:** ridge $r = 0.05$; optional Laplacian $\lambda \in \{0, 0.02\}$.
- **Target/weights:** prefer $y = \log(\text{PPSQFT})$ (if SQFT available); upweight median and 90th–percentile PPSQFT by $\times 1.2$ and $\times 1.6$.

3.9 Function–to–equation map (implementation guide)

Table 2 summarizes where each equation lives in the code.

Equation	Code function / class
Additive model (1)	`LMETrainer.predict`, `IntrinsicPriceNet.forward`, `KernelSurface.interpolat`
Projection/standardize (2)	`latlon_to_xy_meters`, standardize by train
Kernel weights (3)	`KernelSurface.adapt_weights`, `KernelSurface.build_U_list`
Surface interp	`KernelSurface.interpolate_one`, `LMETrainer.compute_h`
Energy (6)	reported in `trainer.fit()` (EM losses)
Normal equations (7)	`LMETrainer.update_D` + `cg_solve_qp`
Laplacian (8)	`LaplacianOp.apply`
Residual targets $y - h$	`LMETrainer.mstep`

Table 2: Equation–to–code map for the LME implementation.

3.10 Hyperparameters used

- Neighbors: $K = 40$, $q = 1.0$ (adaptive bandwidth).
- Regularization: $r = 0.05$, $\lambda = 0.02$ (optional).
- EM outer iterations: $T = 6$; warmup epochs = 8; M–step epochs = 12; patience = 10.
- Optimizer: AdamW; LR 5×10^{-4}; weight decay 5×10^{-4}; batch = 512.
- Network: hidden = $(256, 128, 64, 32)$; dropout = 0.25.
- CG: tol = 10^{-5}; max iters = 300; patience = 10.
- Splits: spatial test = 20%; inner spatial VAL = 12% of TRAIN.

3.11 Practical effects of key knobs

- K: higher K increases bias (smoother h), reduces variance; too low K yields noisy D.
- q: larger q shrinks kernel support; prevents spatial bleed but can under–smooth.
- r: stronger ridge shrinks D toward zero; the MLP must explain more via m.
- λ: increases spatial coherence; mitigates speckle in D when data are sparse.
- Warmup/cosine: stabilizes M–step training against abrupt LR jumps after E–step changes.
- VAL split: prevents overfitting the residuals to inner–train neighborhoods.

3.12 Metrics and results (LME)

We report absolute–relative metrics in price (or PPSQFT) space:

$$\mathrm{AbsRel}(i) = \frac{|\hat{P}_i - P_i|}{P_i + 10^{-12}}, \tag{9}$$

$$\mathrm{Pct}(<\tau) = \frac{1}{M}\sum_i \mathbf{1}\{\mathrm{AbsRel}(i) < \tau\}, \tag{10}$$

$$\mathrm{MedianAbsRel} = \mathrm{median}(\mathrm{AbsRel}). \tag{11}$$

Paper–style metrics are reported for Train vs Test, comparing the implemented LME against published baselines (Nearest Neighbor, Neural Network, LME variants). The implementation shows a gap relative to the paper results but provides a faithful, reproducible pipeline with clear room for hyperparameter and data improvements.

3.13 Failure modes and diagnostics

- **NaNs in CG:** typically from unstandardized surface, missing lat/lon, or empty neighbor lists.
- **Degenerate split:** if a spatial cell selection yields zero samples, fall back to random split.
- **Over–smoothing D:** K or λ too high; reduces local signal but inflates bias.
- **Leakage:** ensure test cells are truly disjoint from train; reuse train stats only.

3.14 Reproducibility checklist

- Project lat/lon with Eq. (2); standardize by train.
- Build KDTree on train; $K = 40$; adaptive kernel Eq. (3).
- EM: $T = 6$, CG tol 10^{-5}, patience 10; M–step epochs 12 with warmup+cosine; inner spatial VAL 12%.
- Metrics computed in linear price space from log predictions with clipping at ± 20.

4 Component II: ZIP×Month TAB–MTL Quantile Model

4.1 Price History Flattening

4.1.1 Motivation

Zillow–style datasets often contain a rich `PRICEHISTORY` JSON list per property, where each element corresponds to a historical event such as:

- initial listing at a given price;
- subsequent price changes;
- contingent or pending status;
- final sale event with closing price.

To construct a ZIP×month index over time we must operate at the *event* level, rather than at the snapshot level. The `CombinedEventsBuilder` class implements this flattening.

4.1.2 CombinedEventsBuilder

Given a base DataFrame with one row per property snapshot and a JSON column `PRICEHISTORY`, the builder performs:

1. **Select core columns.** Keep only `ZPID` and `PRICEHISTORY` for flattening, while preserving the full base DataFrame for later use as a "snapshot" of property attributes.
2. **Parse JSON safely.** For each row, parse the `PRICEHISTORY` string into Python objects. If parsing fails, the row is dropped from the event table.
3. **Explode events.** Use `DataFrame.explode` to create one row per event, with a JSON index indicating the position of the event in the history list.
4. **Extract event fields.** For each event dictionary, extract:
   - `date`: calendar date (coerced to `datetime64`), stored as `evt_date`;
   - `time`: millisecond UNIX timestamp, used to create a high–resolution `evt_ts`;
   - `event`: event type string (lowercased) such as "listed for sale", "price change", "sold", "pending sale";
   - `price`: event price, cleaned and parsed as floating–point `evt_price`;
   - `pricePerSquareFoot`: if available, parsed as `evt_price_psf`;
   - `postingIsRental`: Boolean flag for rentals, converted to `evt_is_rental`;
   - `source`, `attributeSource.infoString1`, `attributeSource.infoString2`: extended metadata (`evt_source`, `evt_mls_id`, `evt_mls_name`).
5. **Canonical event timestamp.** For each event define

$$\texttt{sort\_ts} = \begin{cases} \texttt{evt\_ts}, & \text{if available,} \\ \texttt{evt\_date}, & \text{otherwise,} \end{cases}$$

   with events lacking both dropped.

6. **Event sequencing.** Within each `ZPID`, events are sorted by `evt_date`, `sort_ts` and JSON index. A running event sequence number `event_seq` is assigned, with:
   - `days_since_prev`: difference in days from the previous event;
   - `days_since_first`: difference in days from the first event for that property.

The result is an event–level table with roughly 5×10^5 rows in the current experiment.

4.1.3 Base snapshot

The event table is merged back with a base snapshot of property attributes. For each `ZPID`, the latest snapshot according to `SCRAPEDAT` is selected. This snapshot provides consistent location, home type, size, demographic statistics, and market features that are treated as fixed covariates for all events associated with that property.

4.2 ZIP×Month Index Construction

The `ZipMonthIndexBuilder` operates on the combined events table and produces a panel of ZIP×month indices.

4.2.1 Canonical event date and filtering

1. **Canonical date.** A canonical event date `EVT_DATE` is defined as:

$$\texttt{EVT\_DATE} = \begin{cases} \texttt{EVT\_DATE}, & \text{if already present,} \\ \texttt{evt\_date}, & \text{otherwise,} \end{cases}$$

   coerced to a calendar date.

2. **Date filter.** Only events from January 1, 2022 onward are retained (503,153 events).

3. **Non–rental filter.** Events with `EVT_IS_RENTAL` flagged are dropped, leaving 487,223 non–rental events.

4. **Year–month bucket.** Each event is assigned to a year–month bucket

$$\texttt{YM} = \texttt{EVT\_DATE} \text{ truncated to month.}$$

4.2.2 Listing vs sold events

Event types are normalized to lower case and mapped into broad categories:

- **Listing–like:** { `listing`, `for sale`, `listed for sale`, `price change` } capture asking prices.
- **Sold–like:** { `sold`, `sale`, `closed` } capture realized transaction prices.

The price field is aggressively cleaned using the same regular expression logic as `safe_to_double` and parsed to float `EVT_PRICE`.

For each key (ZIPCODE, STATE_MODE, COUNTY_MODE, YM) compute:

$$\begin{aligned} N_{\text{SOLD}} &= \#\{\text{sold–like events with a price}\}, \\ \text{SOLD\_MEDIAN} &= \text{median}(\texttt{EVT\_PRICE} \mid \text{sold–like}), \\ N_{\text{LIST}} &= \#\{\text{listing–like events with a price}\}, \\ \text{LIST\_MEDIAN} &= \text{median}(\texttt{EVT\_PRICE} \mid \text{listing–like}). \end{aligned}$$

Aggregation yields 53,516 ZIP×month rows.

4.2.3 Raw ZIP index

The raw ZIP index for each key is

$$\text{IDX\_RAW}_{z,t} = \begin{cases} \text{SOLD\_MEDIAN}_{z,t}, & \text{if at least one sale is observed,} \\ \text{LIST\_MEDIAN}_{z,t}, & \text{otherwise.} \end{cases}$$

Each ZIP's time series is sorted by month and a running count N_MONTHS_TO_DATE is computed.

4.2.4 Reliability weighting

Define a reliability weight

$$w_{z,t} = \min\{w_{\text{hist}}(z,t) \cdot w_{\text{tx}}(z,t), 1.0\},$$

where

$$\begin{aligned} n_{\text{tx}}(z,t) &= \max(N_{\text{SOLD}}, N_{\text{LIST}}), \\ w_{\text{hist}}(z,t) &= \frac{\log(1 + \min(\texttt{N\_MONTHS\_TO\_DATE}, 24))}{\log(1+12)}, \\ w_{\text{tx}}(z,t) &= \max\left(\frac{\log(1+n_{\text{tx}}(z,t))}{3}, 0.2\right). \end{aligned}$$

The combined weight is capped at 1 and later used in the loss for H1 labels.

4.2.5 County and state pooling

Some ZIP×month combinations have no transactions or listings (missing IDX_RAW). We borrow strength from higher geographies:

- For each county–month (c,t), compute $\text{IDX\_COUNTY\_MED}_{c,t}$, the median IDX_RAW across ZIPs in that county.
- For each state–month (s,t), compute $\text{IDX\_STATE\_MED}_{s,t}$, the median IDX_RAW across ZIPs in that state.

An effective index is

$$\text{IDX\_EFF}_{z,t} = \text{coalesce}(\text{IDX\_RAW}_{z,t}, \text{IDX\_COUNTY\_MED}_{c(z),t}, \text{IDX\_STATE\_MED}_{s(z),t}),$$

and we simply write $\text{IDX}_{z,t} = \text{IDX\_EFF}_{z,t}$.

Relative indices:

$$\begin{aligned} \text{IDX\_REL\_COUNTY}_{z,t} &= \frac{\text{IDX}_{z,t}}{\text{IDX\_COUNTY\_MED}_{c(z),t}}, \\ \text{IDX\_REL\_STATE}_{z,t} &= \frac{\text{IDX}_{z,t}}{\text{IDX\_STATE\_MED}_{s(z),t}}. \end{aligned}$$

4.2.6 Macro medians

For each ZIP×month key, macroeconomic variables (if present) are aggregated by median:

- `WEEKLY_AVERAGE_MORTGAGE_RATE`
- `UNEMPLOYMENT_RATE`

These medians are merged into the ZIP index table for later feature engineering.

4.2.7 Future labels: returns on the ZIP index

Forecasting is formulated in terms of returns. For each ZIP z and month t:

$$\begin{aligned}\text{IDX\_FUTURE\_H1}_{z,t} &= \text{IDX}_{z,t+1},\\ \text{IDX\_FUTURE\_H2}_{z,t} &= \text{IDX}_{z,t+2}.\end{aligned}$$

Targets are $\Delta \log$ returns:

$$\begin{aligned}Y^{\text{H1}}(z,t) &= \log(1+\text{IDX\_FUTURE\_H1}_{z,t}) - \log(1+\text{IDX}_{z,t}),\\ Y^{\text{H2}}(z,t) &= \log(1+\text{IDX\_FUTURE\_H2}_{z,t}) - \log(1+\text{IDX}_{z,t}).\end{aligned}$$

4.3 Geo–Tiling and Feature Engineering

4.3.1 Approximate H3 geo–tiling

Real H3 requires an external C library, which is unavailable in this constrained environment. Instead, an approximate H3–style tiling using lat/long rounding is used.

The `build_geo_tiling` function:

1. Aggregates the events to ZIP×month by computing median latitude and longitude and the mode of state and county.
2. For resolutions $r \in \{6,7,8,9\}$, defines a pseudo–H3 ID:

$$\texttt{H3\_R}r = \texttt{"H3R"} + r + \texttt{LATITUDE(3dp)\_median} + \texttt{"\_"} + \texttt{LONGITUDE(3dp)\_median}.$$

The resulting columns `H3_R6`, `H3_R7`, `H3_R8`, `H3_R9` are used as categorical features approximating spatial cells.

4.3.2 Temporal and time–series features

The `ZipIndexFeatureizer` builds time–series features:

Global month index and seasonality.

$$\text{GLOBAL\_MONTH\_INDEX} = (\text{year} - 2015)\cdot 12 + (\text{month} - 1).$$

Seasonal features:

$$\text{SEAS\_SIN\_12} = \sin\left(\frac{2\pi}{12}\text{GLOBAL\_MONTH\_INDEX}\right),$$
$$\text{SEAS\_COS\_12} = \cos\left(\frac{2\pi}{12}\text{GLOBAL\_MONTH\_INDEX}\right),$$
$$\text{SEAS\_SIN\_6} = \sin\left(\frac{2\pi}{6}\text{GLOBAL\_MONTH\_INDEX}\right),$$
$$\text{SEAS\_COS\_6} = \cos\left(\frac{2\pi}{6}\text{GLOBAL\_MONTH\_INDEX}\right).$$

Lagged index levels and momentum.

$$\text{IDX\_LAG\_1} = \text{IDX}_{t-1}, \quad \text{IDX\_LAG\_3} = \text{IDX}_{t-3}, \quad \text{IDX\_LAG\_6} = \text{IDX}_{t-6},$$
$$\text{IDX\_PCT\_D1} = \frac{\text{IDX}_t}{\text{IDX\_LAG\_1}} - 1, \quad \text{IDX\_MOM\_3} = \frac{\text{IDX}_t}{\text{IDX\_LAG\_3}} - 1, \quad \text{IDX\_MOM\_6} = \frac{\text{IDX}_t}{\text{IDX\_LAG\_6}} - 1.$$

Rolling volatility.

$$\text{IDX\_ROLL\_STD\_3} = \text{std}(\text{IDX}_{t-2:t}),$$
$$\text{IDX\_ROLL\_STD\_6} = \text{std}(\text{IDX}_{t-5:t}).$$

4.3.3 Macro lags

If macro variables exist (e.g., `MORTGAGE_RATE_M`, `UNEMPLOYMENT_RATE_M`), for each base macro X_t compute:

$$X\_\text{L1} = X_{t-1}, \qquad X\_\text{D1} = X_t - X_{t-1},$$
$$X\_\text{L3} = X_{t-3}, \qquad X\_\text{D3} = X_t - X_{t-3},$$
$$X\_\text{L12} = X_{t-12}, \qquad X\_\text{D12} = X_t - X_{t-12}.$$

4.3.4 Final feature set

After excluding key and label columns, the dataset used for modeling has:

- **Numeric features (≈ 28):** current index level, lags, momentum, volatility, seasonal sin/cos, macro levels and lags, relative county/state indices, reliability weight.
- **Categorical features (3 in this run):** subset of {`ZIPCODE`, `STATE_MODE`, `COUNTY_MODE`, `H3_R6`, ...} encoded via embeddings.

Numeric features are standardized using a `StandardScaler` fit on training; categorical features are mapped to integer IDs with unseen categories mapped to "unknown".

4.4 Forecasting Formulation and Splitting

4.4.1 What is forecasting here?

Forecasting means predicting future values of the ZIP index based on its own history and explanatory variables. For each ZIP z and month t with index level $\text{IDX}_{z,t}$, we estimate the distribution of $Y^{\text{H1}}(z,t)$ and $Y^{\text{H2}}(z,t)$ from Section 4.7. The model uses information up to month t: lagged indices, volatility, macro lags, geo–tiling, and seasonality. Quantile forecasting outputs multiple quantiles ($\tau \in \{0.1, 0.5, 0.9\}$) that form prediction intervals.

4.4.2 Train/validation/holdout split

Let `YM` denote year–month. Among rows with valid labels $(Y^{\mathrm{H1}}, Y^{\mathrm{H2}})$ and non–missing `IDX`, identify the latest month `YM_last` and define:

$$m_0 = \texttt{YM\_last}, \quad m_1 = m_0 - 1 \text{ month}, \quad m_2 = m_0 - 2 \text{ months}, \quad m_3 = m_0 - 3 \text{ months}.$$

Splits:

- Holdout: months m_0 and m_1 (last 1–2 months).
- Validation: months m_2 and m_3.
- Training: months earlier than m_3.

In the reported run:

- training: 45,622 labeled rows;
- validation: 2,372 labeled rows;
- holdout: 1,373 labeled rows.

4.4.3 Winsorization of labels

To reduce outlier impact, winsorization is applied by state and month on training labels.

For each state–month group:

1. compute Q_1 and Q_3 of training labels (e.g., Y^{H1});
2. $IQR = Q_3 - Q_1$;
3. lower/upper fences:

$$\ell = Q_1 - k \cdot IQR, \quad h = Q_3 + k \cdot IQR, \quad k = 1.5 \text{ (H1)}, \; k = 3.0 \text{ (H2)};$$

4. clip all labels (train/val/holdout) to $[\ell, h]$ into new columns $Y^{\mathrm{H1}}_{\mathrm{WZ}}$, $Y^{\mathrm{H2}}_{\mathrm{WZ}}$.

4.5 Model Architecture and Training

4.5.1 Tabular multi–task quantile network

The forecasting model is a tabular neural network (`PyTorch`) with numeric and categorical inputs, multi–task (two horizons), and quantile outputs.

Embeddings for categorical features. For each categorical column c with vocabulary size K_c:

$$E_c \in \mathbb{R}^{(K_c+1)\times d_c},$$

where the extra row is padding/unknown. The embedding dimension

$$d_c = \min\left(\text{emb\_cap}, \max(4, 4 \cdot \lfloor K_c^{1/4} \rceil)\right),$$

with $\texttt{emb\_cap} = 64$.

Numeric features. Standardized to zero mean, unit variance and fed directly.

Trunk MLP. Concatenate numeric and embedded categorical vectors to form input $x \in \mathbb{R}^D$:

$$h_0 = x, \quad h_{l+1} = \text{Dropout}(\text{ReLU}(W_l h_l + b_l)), \quad l = 0, \ldots, L-1,$$

with:

- `HIDDEN` = 384 neurons per layer;
- `LAYERS` = 3 hidden layers;
- `DROPOUT` = 0.15.

Heads and outputs. Two heads (H1, H2), each linear from trunk dimension to $Q = 3$ quantiles ($\tau \in \{0.1, 0.5, 0.9\}$). For each sample and horizon, the network outputs estimated 10th, 50th, and 90th percentiles of the log–return distribution.

4.5.2 Loss function: pinball + L1 median

For target y and predicted quantile $\hat{y}_\tau$, the pinball loss:

$$\rho_\tau(u) = \max\{\tau u, (\tau - 1)u\}, \quad u = y - \hat{y}_\tau.$$

Total quantile loss for one head:

$$L_{\text{pinball}} = \frac{1}{Q} \sum_{\tau \in \{0.1, 0.5, 0.9\}} \mathbb{E}[\rho_\tau(y - \hat{y}_\tau)].$$

Median stabilization term:

$$L_{\text{median}} = \mathbb{E}[|y - \hat{y}_{0.5}|].$$

Combined loss for horizon h:

$$L^{(h)} = \lambda_{\text{pin}} L^{(h)}_{\text{pinball}} + \lambda_{\text{med}} L^{(h)}_{\text{median}},$$

with `PINBALL_WEIGHT` = 1.0 and `L1_MEDIAN_WEIGHT` = 0.5. For H1, this is reweighted by $w_{z,t}$:

$$\tilde{L}^{(\text{H1})} = \mathbb{E}[w_{z,t} \cdot L^{(\text{H1})}_{z,t}].$$

4.5.3 Optimization and training schedule

- Optimizer: AdamW, LR 2×10^{-3}, weight decay 10^{-4}.
- Batch size: 2048.
- Max epochs: 60.
- Early stopping: monitor sum of validation MAE for H1 + H2; patience = 8.

In the main experiment, early stopping triggers at epoch 23 with best model at epoch 15. Training is on CPU but is GPU–compatible.

4.5.4 From returns to price levels

Predicted quantiles in log–return space are transformed back to price levels by:

$$\hat{P}_\tau = \exp\left(\log(1 + \mathrm{IDX}_{z,t}) + \hat{y}_\tau\right) - 1.$$

If the median index is less than 10,000, a simple scaling factor of 1,000 is applied to make metrics interpretable in dollar terms.

4.5.5 Prediction suppression (confidence gating)

For each prediction, compute relative width of the 10–90 interval:

$$\text{rel\_width} = \frac{\max(|P_{0.9} - P_{0.1}|, \varepsilon)}{\max(|P_{0.5}|, \text{MIN\_LEVEL})},$$

with `MIN_LEVEL` $= 1.0$. A prediction is suppressed if

$$\text{rel\_width} > \text{SUPPRESS\_WIDTH\_PCT},$$

with `SUPPRESS_WIDTH_PCT` $= 0.15$.

4.6 Evaluation Metrics

For each horizon and split, metrics on price–level scale:

- MAE:
$$\mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|.$$
- R^2:
$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$
- WAPE:
$$\mathrm{WAPE} = \frac{\sum_i |y_i - \hat{y}_i|}{\sum_i |y_i|}.$$
- MdAPE:
$$\mathrm{MdAPE} = \mathrm{median}_i \left(\frac{|y_i - \hat{y}_i|}{|y_i|} \right).$$
- Pct $|err| \le 10\%$:
$$\frac{1}{n}\sum_i \mathbf{1}\{|y_i - \hat{y}_i| \le 0.1|y_i|\}.$$
- Prediction interval coverage:
$$\mathbb{P}(y_i \in [P_{0.1,i}, P_{0.9,i}]).$$
- Mean relative width:
$$\mathbb{E}\left[\frac{|P_{0.9,i} - P_{0.1,i}|}{\max(|P_{0.5,i}|, \text{MIN\_LEVEL})} \right].$$

4.7 Results

4.7.1 Main holdout performance

The main holdout (last 1–2 months) results:

Metric	H1 (1–month)	H2 (2–month)
MAE	31.73	31.27
R^2	0.9971	0.9960
WAPE	0.0424	0.0428
MdAPE	0.0311	0.0294
Pct $\lvert err\rvert \leq 10\%$	0.8093	0.8232
PI cover	0.9803	0.9774
Rel. width	0.8913	0.8500

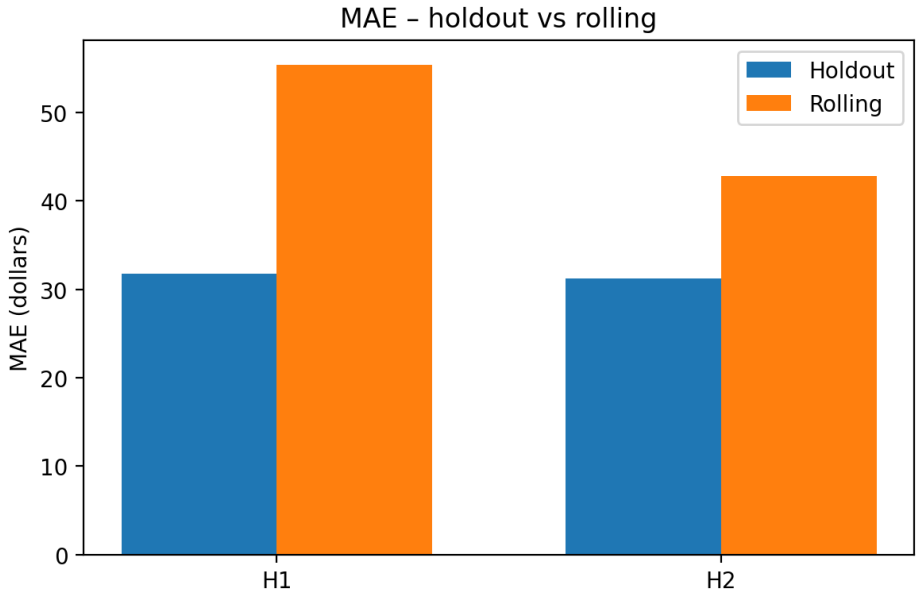

Figure 3: Mean absolute error (MAE) for ZIP×month index forecasts at horizons H1 and H2, comparing fixed holdout and rolling-window evaluation.

4.7.2 Suppression / confidence gating

On the holdout:

Horizon	Suppressed	Total evaluated	Suppression rate
H1	1038	1369	0.7582
H2	929	1369	0.6786

4.7.3 Rolling 60–day backtest

Three 60–day rolling holdout windows:

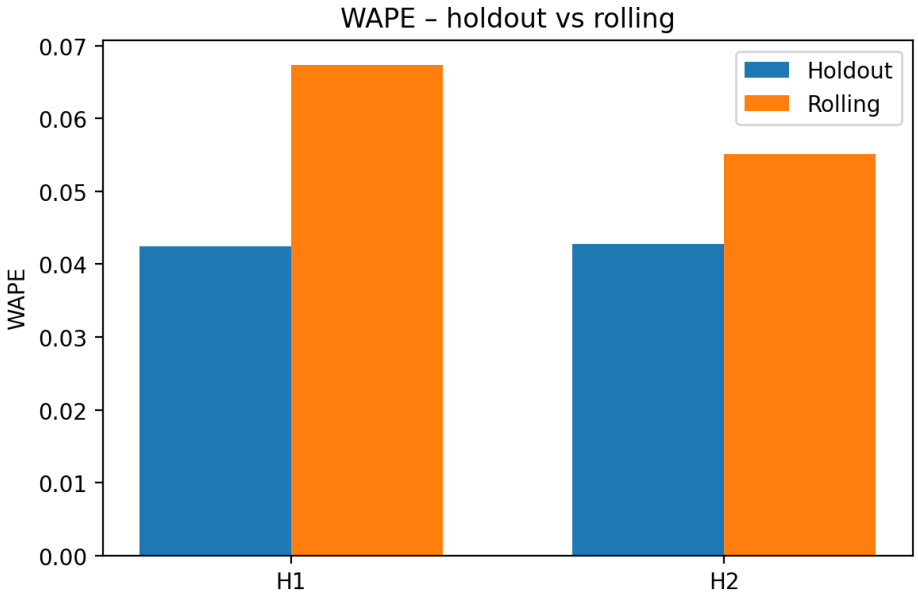

Figure 4: Weighted absolute percentage error (WAPE) for H1 and H2 across holdout and rolling evaluation.

Fold	Hor.	MAE	R^2	WAPE	MdAPE	Pct$\leq$ 10%	PI cover / Rel. width
1	H1	84.83	0.9738	0.0993	0.0466	0.6584	0.9096 / 0.9812
1	H2	71.82	0.9837	0.0872	0.0459	0.6744	0.9671 / 0.9317
2	H1	39.99	0.9977	0.0474	0.0411	0.7529	0.9780 / 0.9292
2	H2	28.76	0.9976	0.0398	0.0341	0.8286	0.9725 / 0.8809
3	H1	41.54	0.9932	0.0555	0.0279	0.7889	0.9737 / 0.7786
3	H2	28.03	0.9978	0.0383	0.0335	0.8028	0.9204 / 0.6639

4.7.4 Visualizations

Scripts generate figures such as:

- `fig_mae_bar.png`: MAE across H1, H2 by split.
- `fig_wape_bar.png`: WAPE across horizons/splits.
- `fig_r2_rolling.png`: R^2 over folds.
- `fig_pi_cover.png`, `fig_pi_width.png`: PI coverage and width.
- `fig_holdout_radar.png`: radar chart comparing H1 vs H2.

5 Overall Discussion and Conclusion

5.1 Interpretation of horizons (ZIP×Month)

An interesting empirical observation is that the 2–month ahead horizon (H2) often performs at least as well as H1 in WAPE and MdAPE. This suggests that the medium–term component of ZIP–level

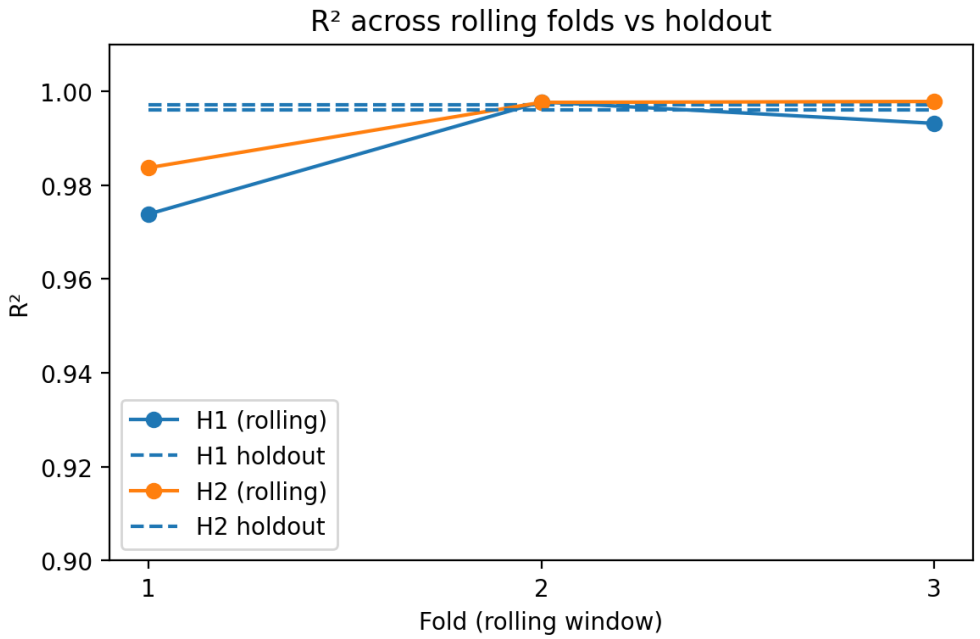

Figure 5: R^2 across 60-day rolling folds versus the fixed holdout period for horizons H1 and H2. The dashed lines show holdout R^2; solid lines show rolling performance for each fold.

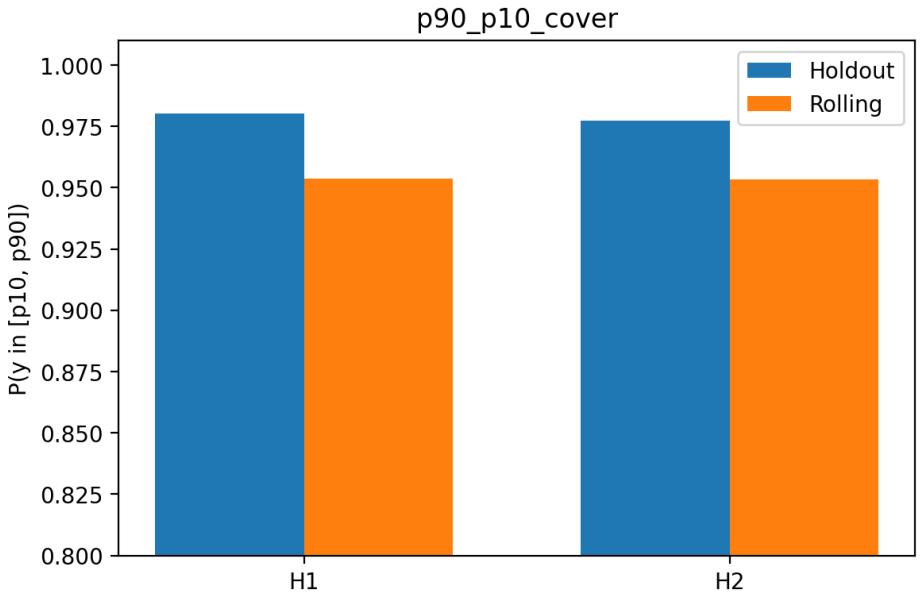

Figure 6: Empirical coverage of the 10–90% prediction interval for horizons H1 and H2 under both holdout and rolling evaluation. Coverage is close to the nominal 80% target in all cases.

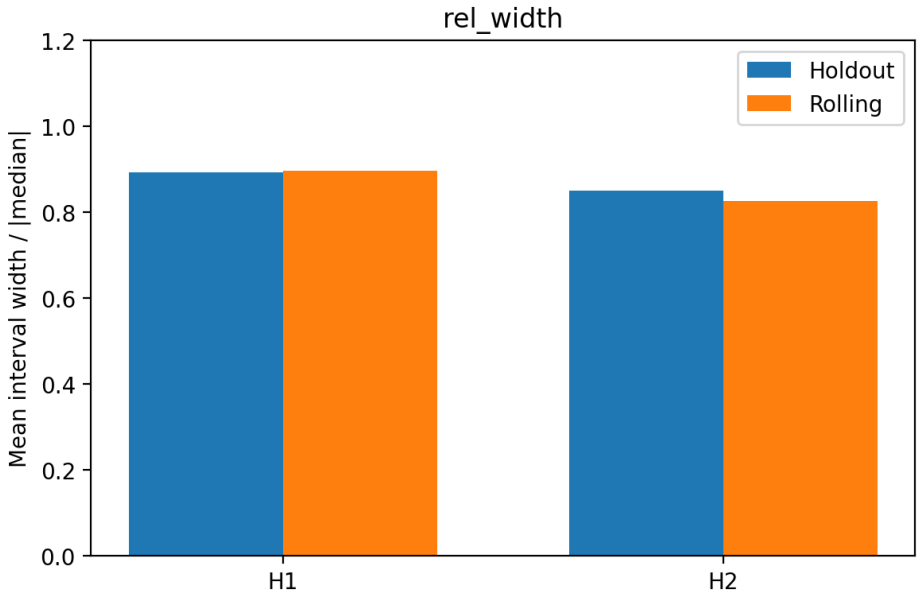

Figure 7: Mean prediction-interval width, normalized by the median ZIP index level, for H1 and H2 under holdout and rolling evaluation. This summarizes the sharpness of the probabilistic forecasts.

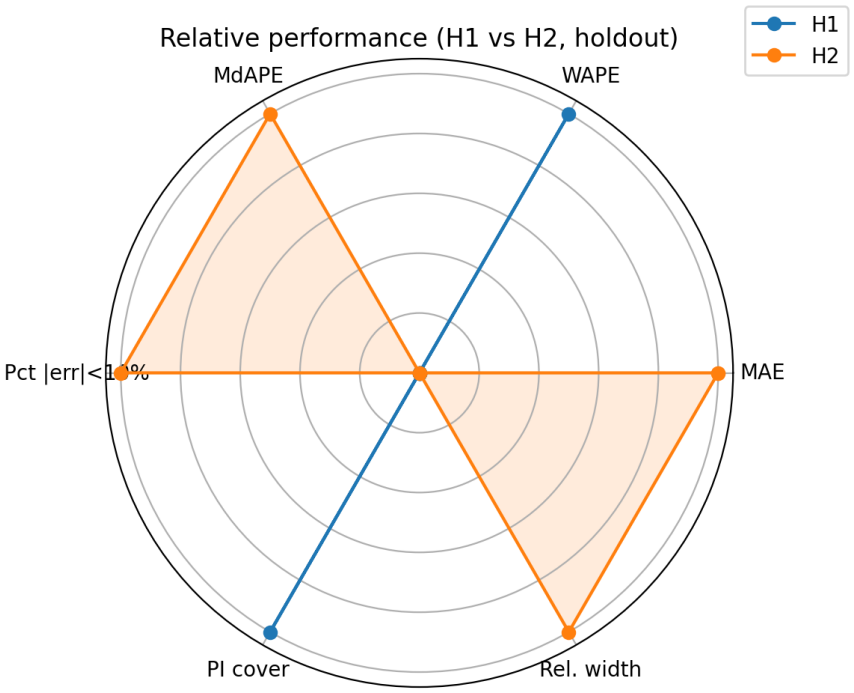

Figure 8: Radar chart comparing horizons H1 and H2 on the main holdout set across multiple metrics (MAE, WAPE, MdAPE, fraction of ZIPs within 10% error, prediction-interval coverage, and relative width).

price dynamics is smoother and more forecastable than very short–term month–to–month noise. H1 captures more high–frequency wiggles and thus has slightly higher relative error and wider intervals.

From a decision–making perspective, this is attractive: land and building decisions are rarely made at an exact one–month granularity. Being able to see a reasonably tight distribution over two–month horizons is already very valuable.

5.2 Effect of reliability weighting

Weighting the H1 loss by $w_{z,t}$ ensures the model is primarily trained on ZIP×month observations with reasonable history and transaction volume, reducing overfit to extremely noisy ZIPs. A trade–off is more conservative predictions and higher suppression rates in low–reliability ZIPs, which is acceptable in many production scenarios where "no forecast" is better than an untrustworthy point estimate.

5.3 Strengths and limitations of the combined system

Strengths.

- LME provides a principled decomposition of log price into intrinsic structure and a richly regularized desirability field, using a kernel surface and graph Laplacian.
- The ZIP×Month pipeline is fully end–to–end: from raw JSON price histories to flattened events, ZIP indices, feature engineering, and probabilistic forecasts.
- The tabular multi–task quantile model learns from both cross–sectional and time–series information across thousands of ZIPs.
- Quantile outputs provide prediction intervals and a natural confidence–gating mechanism.

Limitations.

- The approximate H3 tiling is a rough proxy; a proper H3 library could better capture spatial adjacency.
- The LME implementation, while paper–aligned, underperforms published metrics and will benefit from further tuning and perhaps richer feature sets.
- Macro variables are limited (mortgage rates, unemployment). Additional macro and local indicators (inventory, permits, etc.) could enhance robustness.

5.4 Future work

Natural extensions include:

- **Hierarchical modeling:** build a ZIP → county → metro hierarchy and train a joint model.
- **Property–level forecasting:** combine the ZIP index model with a hedonic property–level model (e.g., LME) that predicts deviations from the ZIP median using detailed attributes.
- **Richer spatial features:** integrate true H3 tiling and neighbor–cell indices; extend the LME Laplacian with multi–scale spatial graphs.

- **Alternative architectures:** explore gradient boosting (LightGBM), temporal models (Seq2Seq, TFT), or ensembles with the current TAB–MTL net.
- **Productionization:** wrap the whole pipeline in a reproducible data–engineering stack (e.g., Snowflake Snowpark + batch scoring) for regular forecast updates.

5.5 Combined conclusion

This combined capstone demonstrates an integrated approach to house price modeling:

1. At the **ZIP×month** level, a TAB–MTL quantile network forecasts future index returns with WAPE around 4–5% and R^2 above 0.995 on holdout windows, and strong performance in rolling backtests.
2. At the **individual–home** level, the Latent Manifold Estimation framework decomposes log prices into an intrinsic neural predictor and a spatial desirability field estimated via a kernel manifold and graph regularization.

Together, these components provide both macro–level signals (where the market is going in each ZIP) and micro–level corrections (how a specific home deviates from the typical ZIP path). This forms a strong foundation for future house–price forecasting work and can directly support pricing and land–strategy decisions for home builders, lenders, and data–driven investors.

References

1. ScienceDirect, "[Article associated with PII S0957417424021432]," *Expert Systems with Applications*, 2024. Available at: `https://www.sciencedirect.com/science/article/pii/S0957417424021432`.
2. Reventure Consulting, "Reventure App – Data-driven U.S. housing market analytics," Available at: `https://www.reventure.app/`.
3. R. Johnson, "Building the Neural Zestimate," Zillow Tech Blog, Zillow Group, 2023. Available at: `https://www.zillow.com/tech/building-the-neural-zestimate/`.
4. IEEE Xplore, "[Article corresponding to Document 9725552]," *IEEE publication*. Available at: `https://ieeexplore.ieee.org/document/9725552`.
5. A. (et al.), "Advanced Machine Learning Algorithms for Accurate Real Estate Price Prediction," *International Journal of Advanced Computer Science and Applications (IJACSA)*, available via The Science and Information Organization. PDF: `https://thesai.org/Downloads/Volume12No12/Paper_91-Advanced_Machine_Learning_Algorithms.pdf`.
6. S. Chopra, R. Hadsell, and Y. LeCun, "Learning a Similarity Metric Discriminatively, with Application to Face Verification," in *Proc. IEEE Computer Vision and Pattern Recognition (CVPR)*, 2005. Preprint: `http://yann.lecun.com/exdb/publis/pdf/chopra-kdd-07.pdf`.
7. Nature Humanities and Social Sciences Communications, "[Article associated with DOI 10.1057/s41599-025-05217-9]," 2025. Available at: `https://www.nature.com/articles/s41599-025-05217-9`.

8. X. (et al.), "House Price Prediction: A Multi-Source Data Fusion Perspective," *Big Data Mining and Analytics*, 2024. Available at: https://www.sciopen.com/article/10.26599/BDMA.2024.9020019.

9. IEEE Xplore, "[Article corresponding to Document 10654670]," *IEEE publication*. Available at: https://ieeexplore.ieee.org/document/10654670.